SARBLOH: Structured Geometric Perception and Speculative Code Execution for Autonomous ARC-AGI-3 Agents

Harsidak Singh Banwait
Independent Researcher
harsidak.singh.banwait.13@gmail.com

Abstract

ARC-AGI-3 evaluates fluid intelligence of AI or AI based systems by utlizing interactive game and how few actions it needs, relative to a human, to master such unknown environments AI has never seen. Such enviornments which are unknown to LLM, understanding or perception becomes crucial. While LLMs can learn to play games with a few-shot prompt, they struggle with spatial grounding. When SOTA models are shown raw 64×64 integer grids, it burn hundreds of actions rediscovering mechanics a human grasps at a glance. We present SARBLOH AGENT, an agent built around a deterministic host-side perception engine and a code-as-action runtime. In $\approx$35 ms per frame, the engine produces three aligned views: an object-centric state with identity tracking across steps, a briefing that separates verifiable geometric invariants from affordance hypotheses paired with one-action tests, and an annotated spatial map. The agent writes Python world models and action plans over these views, supported by an append-only context that reuses 98.4% of the prefix cache and a priority scheduler that shares one GPU across 25 concurrent games. In a controlled rubric, structured perception raises affordance identification from 3/28 to 25/28 while cutting input tokens by 89.4%. On one RTX PRO 6000, SARBLOH AGENT clears 143 of 183 public levels in 8.2 hours and scores 17.01 on the hidden Kaggle evaluation.

Code: https://github.com/Harsidak/SARBLOH

1 Introduction

Human visual cognition exhibits remarkable inductive fluidity when dealing with novel interactive environments. Observation of an unfamiliar task for less than a hundred milliseconds allows humans to absorb fundamental physical and operational elements. Identifying agent embodiment, understanding spatial boundaries, recognizing temporal and resource gauges, and deducing final goal affordances can occur without the need for explicit complex manuals (Spelke and Kinzler 2007). While these abstractions are inductive and subject to revision, they provide the necessary structural scaffolding that enables humans to solve complex tasks.

The ARC-AGI-3 benchmark (ARC Prize Foundation 2026a) was established to evaluate this core facet of artificial general intelligence: the efficiency of novel skill acquisition under strict epistemic uncertainty (Chollet 2019). Unlike static visual reasoning benchmarks, ARC-AGI-3 requires an agent to operate in a sequential environment that includes an undisclosed series of compositional levels. In this kind of environment, the agent receives no natural language directives, action semantic descriptions, or termination conditions. Furthermore, the benchmark evaluates performance through Relative Human Action Efficiency (RHAE), where the earned score scales quadratically with the ratio of human baseline actions to agent actions. Consequently, each exploratory or misdirected action incurred by an agent penalizes its final score quadratically, rendering brute-force trial-and-error catastrophically inefficient.

When multimodal models are deployed in these environments, their primary failure mode is visual grounding and spatial representation. When an environment frame is serialized into an unparsed matrix of discrete integer tokens, a state-of-the-art (SOTA) reasoning model has to use tens of thousands of tokens to reconstruct the dynamics of these environments—such as connectivity, containment, and coordinate alignment. In preliminary controlled investigations (Section 3), exposing a frontier model to raw 64×64 integer grids resulted in over 200–300 exploratory actions on the navigation task, leading to exhaustion of the action budget and task failure. In stark contrast, providing the same model with a visually aligned rendering and structural guidance enabled direct task resolution in 6 to 12 actions. Geometric briefing enabled direct task resolution in 6 to 12 actions. On interactive abstract reasoning benchmarks, perceptual structuring is the primary operational bottleneck.

This visual bottleneck binds with acute severity under the rigorous constraints of the offline Kaggle ARC Prize track. In this setting, candidate agents are evaluated on undisclosed, held-out game environments without external network connectivity. Computation is strictly bounded to a single NVIDIA RTX PRO 6000 GPU (96 GB VRAM) for a total execution window of nine hours across all evaluated games, and all model weights must be open-source. Under these stringent computational and temporal boundaries, an agent cannot afford iterative self-correction or excessive reflective generation over poorly formatted observations; it requires high-fidelity, deterministic perceptual abstractions that minimize token latency and preserve action efficiency.

In this work, we present SARBLOH AGENT, an autonomous open-weight agent architecture designed specifically for

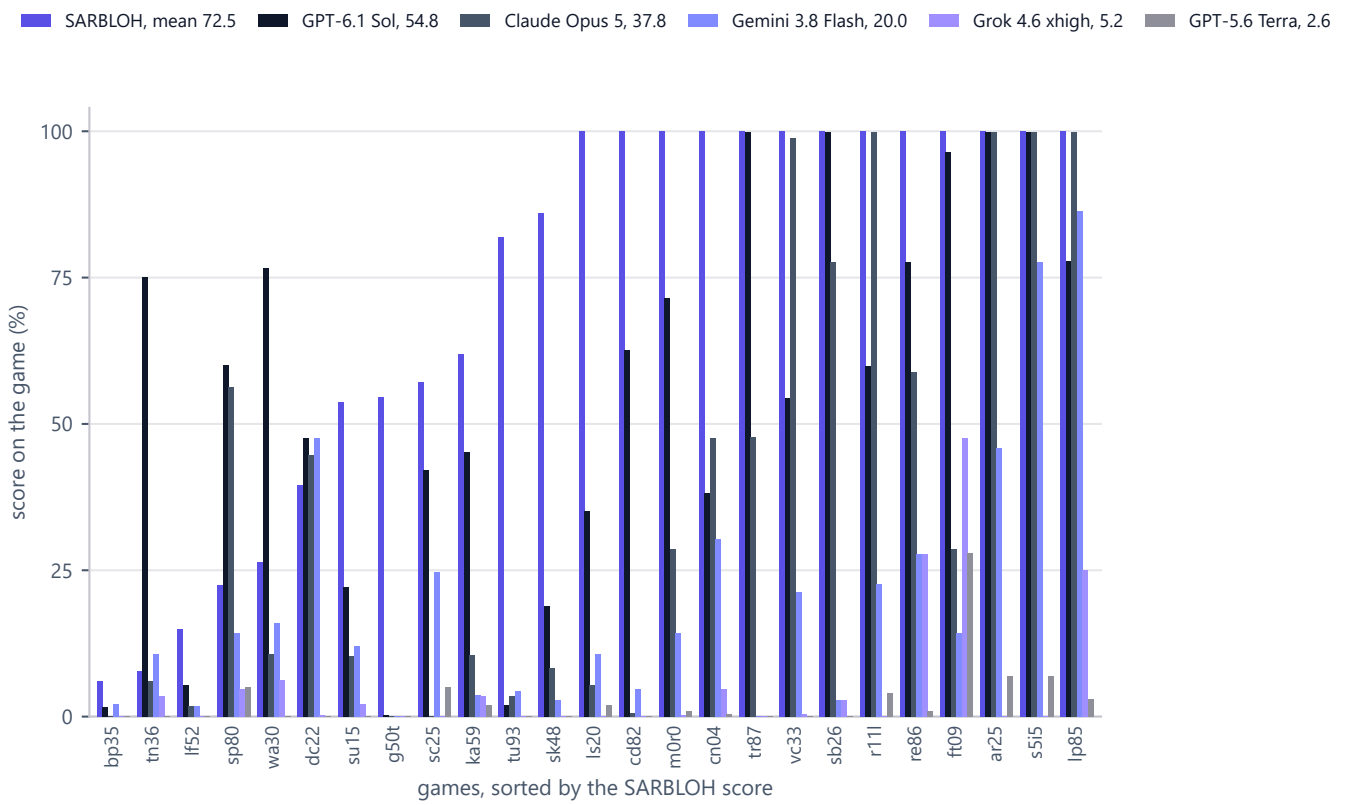


Figure 1: Score on the 25 public ARC-AGI-3 games. One group of bars per game, sorted by the SARBLOH score: the first (indigo) bar is SARBLOH with an open-weight Qwen3.8-Flash-Next model on one GPU, the other five are frontier models. The mean over the 25 games is 72.5 for SARBLOH, 54.8 for GPT-6.1 Sol, 37.8 for Claude Opus 5, 20.0 for Gemini 3.8 Flash, 5.2 for Grok 4.6 xhigh and 2.6 for GPT-5.6 Terra. SARBLOH reaches 100% on 13 games. These are public-game scores, which are not comparable to the hidden-set score in Table 1.

resource-constrained, high-efficiency interactive reasoning on ARC-AGI-3. At the core of SARBLOH AGENT is a zero-parameter, host-side *perception engine* that operates in $\mathcal{O}(HW)$ algorithmic time ($\approx$35 ms per frame), transforming each raw grid state into three structurally synchronized modalities: (i) an object-centric symbolic representation with stable bipartite identity tracking across temporal transitions, (ii) a geometric briefing that isolates mathematically proven spatial invariants (BFS lattice reachability, topological containment, midpoint tethers, and dihedral symmetries under $D_4 \times \mathbb{Z}^+$) from affordance hypotheses paired with unit verification actions, and (iii) an affordance-annotated visual map.

Surrounding this perceptual foundation, SARBLOH AGENT integrates an execution-grounded planning engine operating exclusively via a persistent Python REPL. Rather than issuing myopic primitive actions, the agent synthesizes programmatic world models, explores trajectory trees via algorithmic graph search, and commits multi-step speculative execution paths through a unified primitive that continuously monitors environmental invariants and halts upon sensory prediction discrepancies. To maximize inference throughput on our single-GPU budget, we serve Qwen3.8-Flash-Next—a 125B parameter Mixture-of-Experts architecture activating 6B parameters per token—quantized via AutoRound W4A16 and served using SGLang with EAGLE speculative decoding. By enforcing a strictly monotonic, append-only context protocol, the runtime achieves a 98.4% prompt prefix-cache hit rate across hybrid linear-recurrent and attention layers, while a utility-driven priority scheduler dynamically multiplexes the GPU across 25 concurrent games based on expected marginal RHAE gains.

2 Preliminaries and Related Work

2.1 Environment

An ARC-AGI-3 environment (ARC Prize Foundation 2026a) is an uninstructed, discrete-time game made of an ordered sequence of levels $\{1, \ldots, n\}$. At each step t the agent sees a frame $g_t \in \{0, \ldots, 15\}^{H \times W}$ with $H, W \leq 64$, the episode state $s_t \in \{\texttt{NOT_FINISHED}, \texttt{WIN}, \texttt{GAME_OVER}\}$ and the number of levels cleared. It then picks an action from

$$\mathcal{A} = \{\texttt{ACTION1}, \ldots, \texttt{ACTION5}, \texttt{ACTION6}(x, y), \texttt{ACTION7}, \texttt{RESET}\}, \quad (1)$$

where ACTION1–ACTION4 are directional, ACTION5 and ACTION7 are game-specific, and ACTION6(x, y) is a click at pixel (x, y). Nothing is explained up front: what the actions

do, which object the agent controls and what wins the level must all be inferred from transitions. Early levels introduce single mechanics and later levels compose them (Spelke and Kinzler 2007).

Games are scored by *Relative Human Action Efficiency* (RHAE). Let h_l be the human baseline for level l (the upper-median human action count) and a_l the agent's action count. Then

$$S_l = \min\left(1.15,\ \left(\frac{h_l}{a_l}\right)^2\right), \qquad E = \frac{\min\left(\sum_{l=1}^{k} l,\ \sum_{l=1}^{n} l\, S_l\right)}{\sum_{l=1}^{n} l}, \tag{2}$$

where $k \le n$ is the number of levels cleared, and the benchmark score is the mean of E over games. Three consequences drive our design. Extra actions are penalized quadratically, so an agent that needs twice the human count keeps only 25% of the credit. Thinking and host-side computation are free, and only actions sent to the environment count in a_l, which favours planning in code over trial and error. And the weight l pushes the score toward later levels.

Offline evaluation and resource bounds. The official Kaggle track runs a submitted notebook on held-out games with no network access. It gets one NVIDIA RTX PRO 6000 (Blackwell, 96 GB VRAM), 176 GB of host RAM and a nine-hour wall-clock limit for the whole suite, and all weights must be open and packaged with the notebook. The budget is therefore decode throughput, since reasoning is bounded by tokens per second, and the length of prompt that can stay in the warm prefix cache without eviction or recomputation.

2.2 Related Work

Static Inductive Program Synthesis. Early static ARC benchmarks (Chollet 2019) were dominated by symbolic search over expressive, hand-crafted Domain-Specific Languages (DSLs) (Wind 2020). Subsequent research expanded symbolic synthesis through abstract reasoning graph architectures (Xu, Khalil, and Sanner 2022), hierarchical library learning (Ellis et al. 2020), and Minimum Description Length (MDL) inductive biases (Ferré 2025). In the ARC Prize 2024 competition, state-of-the-art approaches transitioned toward neural synthesis: test-time training (TTT) on augmented task instances (Akyürek et al. 2024; Chollet et al. 2024), unified inductive-transductive program induction (Li et al. 2024a), and ultra-compact recursive neural architectures (Jolicoeur-Martineau 2025). However, these static methodologies are fundamentally transductive: they map small sets of paired input-output demonstration grids to a single static solution grid, operating entirely without interactive environmental feedback or sequential action execution.

Interactive Agents and World Models. Interactive abstract reasoning demands an entirely distinct agentic paradigm. General large language model agents typically interleave textual reasoning and tool execution (Yao et al. 2022) or incrementally construct modular libraries of executable code skills (Wang et al. 2023). In the context of ARC-AGI-3, several frontier systems have emerged. *Prime Agent* (Karten et al. 2026) equips a frontier model with a persistent IPython REPL; our initial agent harness originated as an open-weight port of this execution loop. *Executable World Models* (Rodionov 2026) inductively synthesizes executable Python simulators of game transitions, verifies state invariants against historical traces, and performs trajectory search within the simulated model, drawing inspiration from WorldCoder (Tang, Key, and Ellis 2024) and code-based world models (Lehrach et al. 2025). Similarly, *Tycho* (Lehmann 2026) infers programmatic world models, while graph-based exploration frameworks (Rudakov, Shock, and Cowley 2025) rank potential actions via visual salience without parameter updates. *VISTA* (Han et al. 2026) introduces a multimodal visual harness featuring lossless frame retention. Crucially, all of these systems depend on unbounded, proprietary frontier API endpoints and are evaluated exclusively on the public game distribution, precluding their deployment in offline, resource-bounded environments.

Symbolica's Arcgentica. Symbolica's *Arcgentica* (Knutsen and Klein 2026) is the closest frontier-API relative of our harness. It is built on the open-source Agentica SDK, which gives each agent a persistent Python REPL and lets it spawn sub-agents that receive only the state they need, instead of the full history. On launch day Symbolica reported an unverified 36.08% on the 25 public games with Claude Opus 4.6 at high effort, passing 113 of 182 playable levels and fully solving 7 games, at a stated cost of $1,005 (Symbolica AI 2026). The score was not certified by the organizers and Symbolica describes the harness as not specific to ARC-AGI. Arcgentica shares two ideas with SARBLOH: reasoning inside a persistent code environment, and keeping context small. It does so by delegating to sub-agents, whereas we keep one append-only context and drain it deterministically (Section 4). Its score is also on the contaminated public games with unbounded API calls, so it is not comparable to a hidden-set offline score.

The Open-Weight Offline Frontier. The direct baseline class for SARBLOH consists of open-weight systems evaluated under the official Kaggle offline protocol (Table 1). The Milestone #1 benchmark (June 2026) was achieved by Tufa Labs' *Duck* harness (Tufa Labs 2026; ARC Prize Foundation 2026b). The Duck framework deploys Qwen 3.6 27B locally within an interactive Python execution environment that exposes environment state variables, ingests observations via composite ASCII grids and visual renderings, and evicts historical context windows to permit continuous execution. Its creators established that rigid, hand-crafted agent toolkits degraded performance compared to flexible, unconstrained code execution. Subsequent second- and third-place systems utilized Gemma-4-31B as a visual action generator predicting JSON-formatted action sequences directly from annotated frames (ARC Prize Foundation 2026b). As summarized in Table 1, SARBLOH establishes a new state of the art in this offline open-weight domain.

	Hidden Kaggle set (offline, 1 GPU, 9 h)			Public set (frontier APIs, unbounded)				
	Sarbloh this work	**Duck** Milestone #1, 1st	**Reki** Milestone #1, 2nd	**Prime Agent** open harness	**Arcgentica** Symbolica	**AVO** NVIDIA	**Provider Adapters** GPT-5.5	**Provider Adapters** Gemini 3.8 Flash
Backbone	Qwen3.8-Flash-Next (MoE)	Qwen 3.6 27B	Gemma-4-31B	Claude Opus 5	Claude Opus 4.6	Opus 5 / GPT-5.6 Sol	GPT-5.5	Gemini 3.8 Flash
Open weights	✓	✓	✓	–	–	–	–	–
Score	**16.53**	**1.21**	**0.87**	**95.5**	**36.1**†	**100.0**†	**99.99**†	**≈99.9**†

Table 1: ARC-AGI-3 systems under two evaluation protocols. The left group runs offline on the hidden Kaggle games with open weights on one GPU; the right group calls frontier APIs on the public games. The two scores are not directly comparable. †Reported by the system's authors on the public games and not certified by the organizers; the Provider Adapters scores are as reported to us, and the Gemini figure is approximate. No adapter result for Claude is available.

3 Motivation: Isolating the Visual Grounding Bottleneck

Prior to constructing our agent architecture, we conducted a sequence of controlled ablation experiments designed to isolate the exact computational failure modes where autonomous language model agents squander their action budgets in interactive reasoning tasks.

The Representation Ladder. We evaluated a state-of-the-art frontier model (Gemini 1.5 Pro) on the inaugural level of `ls20`—a representative ARC-AGI-3 environment featuring a discrete keyboard-navigated maze with directional barriers, an inventory key, a locking mechanism, and a dynamic action gauge. Holding the model, prompt instructions, and underlying environment strictly constant, we systematically modulated only the modality and structure of the input observation (Figure 2).

When provided solely with the raw 64×64 integer matrix and the valid action enumeration, the model expended between 200 and 300 exploratory actions, repeatedly colliding with invisible topological boundaries and ultimately exhausting the action budget without reaching the goal state. Serializing spatial lattices into discrete token sequences destroys spatial adjacency priors: evaluating topological reachability across 4,096 integer tokens requires quadratic attention across distant positions in the sequence, an operation that autoregressive attention heads fail to execute reliably (Rahmanzadehgervi et al. 2024).

Conversely, supplying the identical model with a rendered 2D image of the identical board immediately reduced action expenditure to 12–24 actions—an operational speedup exceeding an order of magnitude achieved purely by aligning the state representation with visual pre-training priors. Augmenting the visual frame with a structural decomposition of board components (delineating static terrain, dynamic gauges, and color parity invariants) further condensed the trajectory to 12 actions. Finally, prompting the model to synthesize programmatic search scripts within an execution sandbox yielded optimal completion in exactly 6 actions. The primary performance leap stems from perceptual grounding rather than scale of reasoning.

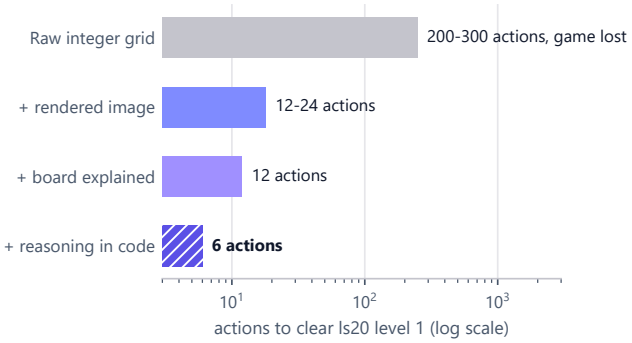


Figure 2: Controlled ablation of input observation modalities on the first level of `ls20`, evaluated using an identical frontier reasoning model. Serializing the environment as raw integer tokens induces catastrophic exploration failure (200–300 actions, exceeding budget). Providing a rendered visual frame reduces required actions by over an order of magnitude ($14\times$ speedup), while augmenting the state with deterministic geometric relations and programmatic execution achieves super-human efficiency (6 actions versus human baseline $\approx$10).

The Oracle Briefing: Decoupling Perception from Planning. To determine whether agent failures originate from deficiencies in high-level policy planning or low-level perceptual parsing, we performed an oracle comprehension experiment. The model was tasked with solving an advanced level of `ls20` under two conditions: (i) providing visual screenshots alone, and (ii) augmenting visual screenshots with an "oracle briefing"—a concise, structured natural language annotation specifying inferred action semantics, goal criteria, mechanical dependencies, and identified on-screen entities.

Without the briefing, the agent squandered its entire ac-

tion allocation exploring inert visual artifacts and failed to resolve the level. In the presence of the structural briefing, the model formulated an optimal plan and cleared the level in 6 actions, substantially outperforming the author's initial blind playthrough ($\approx$10 actions). This empirical divergence establishes a vital architectural insight: modern reasoning backbones possess highly capable sequential planning and programmatic search capabilities, but their execution remains bottlenecked by inaccurate visual state induction. If an agent can be provided with high-fidelity semantic and geometric grounding, its downstream planning executes with near-optimal efficiency.

The Failure of Sparse Coordinate Deltas. In resource-constrained settings, a common optimization is transmitting only sparse coordinate deltas (i.e., the set of grid cells modified between step $t-1$ and t) to minimize prompt token consumption. While computationally economical, our empirical tracking revealed that coordinate deltas induce severe perceptual myopia. In ARC-AGI-3 environments, local agent actions routinely trigger non-local, global state transitions: for example, stepping onto a localized trigger tile in `ls20` instantaneously alters an icon situated in a distant status panel. When an agent receives only localized coordinate updates, it systematically overlooks global mechanical coupling, hallucinating that its action produced no operational side-effects. Maintaining holistic whole-frame visual representations is therefore mandatory to preserve causal tracking across the entire visual field.

Core Design Principles of the Perception Engine. Because oracle briefings cannot exist for undisclosed, held-out environments during official evaluation, an autonomous agent must construct its own structural abstractions dynamically. ARC-AGI-3 environments are strictly grounded in human core knowledge (Spelke and Kinzler 2007), which is predominantly geometric and topological: connectivity, spatial enclosure, lattice-constrained reachability, structural symmetry, and affine transformations.

These properties admit exact, deterministic algorithmic verification. In contrast, operational semantics (e.g., identifying which dynamic component represents the agent embodiment, or what constitutes the terminal win condition) are fundamentally latent under partial observability and can only be induced through empirical interaction.

These findings dictate the central architectural axiom governing SARBLOH:

> *Compute mathematically certain geometric invariants deterministically via zero-parameter host code; formulate ambiguous operational roles as explicit probabilistic hypotheses; and equip every hypothesis with a minimal-cost, single-action falsification test.*

By enforcing this strict dichotomy, the agent eliminates visual hallucination over spatial facts, while ensuring that any incorrect operational hypothesis is refuted through a single probing action, entirely averting catastrophic quadratic exploration penalties.

4 Methodology

Perception is the part of SARBLOH that we developed most deliberately, so most of this section is about it: where it started (Section 4.1), the eight designs we compared in a perception lab (Section 4.2), what survived a held-out check (Section 4.3), and what the final engine computes (Section 4.4). The last subsection (Section 4.5) shows how the engine sits inside the whole agent (Figure 6).

4.1 Where Perception Started

The first perception module, which we call the *Legacy Encoder*, pushed a new observation into the context after every transition. It had four parts: the frame as a PNG in the ARC palette ($4\times$ nearest-neighbour, 256×256 pixels); a lettered board with one letter per colour, shown whole at the start of a level and afterwards only as the changed region with a 3-cell margin and numbered rows and columns; a segmentation list of 4-connected objects with id, colour, shape hash, bounding box, containment and adjacency, with HUD bars along the frame edge flagged; and one change line per transition ("`obj 4 moved dx=0 dy=-5; obj 9 vanished; 2 cells recoloured`"). It cost about 3,500 text tokens per frame.

Three early probes on the first level of `ls20` shaped it. They are qualitative, were logged from the author's own runs, and have no held-out score.

- **Whole-frame picture versus cell deltas.** Sending only the changed cells is cheaper, but some actions change the game globally (stepping onto the modifier in `ls20` does), and a cell-delta view hides that. A whole-frame picture kept the global effect visible.
- **Picture plus segmentation text.** The model understood the frame precisely only when the segmentation list and the lettered board came beside the picture: the picture carries the global layout, the text carries exact positions.
- **Oracle briefing.** A frontier model played the same level twice, once from screenshots and the action space alone and once with a briefing we wrote by hand (action meanings, goal, mechanics, on-screen components). Without the briefing the game was wasted; with it the level was cleared in about six actions, against about ten for the author. Execution was adequate and understanding was the bottleneck (the ladder of Figure 2, Section 3).

The last probe set the target: an engine that writes the briefing itself, from the frame alone, for a game it has never seen.

4.2 The Perception Lab: Eight Ways to Read One Frame

We compared eight ways of presenting the same frame against the Legacy Encoder, one Python file each. All of them share one front end (screenshot, then an exact 64×64 palette grid, then the object list), so the only thing that changes is what is computed on top of the objects and how it is shown. The two test frames are the first levels of `ls20` (keyboard controls) and `r11l` (clicks). Understanding was graded against an answer key written and checked in the engine before any presentation ran: seven items per frame (spatial boundaries, mechanical linkages, agent identity, goal,

gauges and so on) at two points each, 28 points in all, plus an intuition grade from 1 to 5. The kill rule was fixed in advance: drop a presentation that scores no better than the Legacy Encoder (3/28), or whose claims are wrong on about half or more of the held-out games.

- **Exploded Inventory** (3/28) pulls every object out of the frame, enlarges it, spaces the objects apart and stacks identical copies as $\times N$. The list is clean but attaches no meaning to anything.
- **Surprise Map** (4/28) ranks cells by how unpredictable their 3×3 neighbourhood is and flags broken symmetry. About 90% of the frame is predictable texture, and the map caught the lopsided plus on `ls20`, but little else.
- **Visual Routines** (15/28), after Ullman, computes reachability by flood fill with step counts, what lies inside what, line tracing to attached objects, midpoints and alignment. On `r11l` it found the midpoint tether and the ring that fits a 5×5 object.
- **Look-alike Threads** (6/28) joins objects that are equal under rotation, flip, scale or recolouring and names the transform. On `ls20` it showed that the inventory icon is the goal shape turned 90° and drawn at twice the size.
- **Foresight Strip** (12/28) imagines the next frame for each action from simple movement priors. On `ls20` it predicted all four moves and the gauge correctly, and on `r11l` its three-click plan solved the level.
- **Blueprint** (8/28) redraws the frame as a schematic: floor, walls, tile lattice, named shapes and the HUD moved to a side panel. It is the best map for navigation and says nothing about meaning.
- **Role Ledger** (20/28) guesses a role for every object (agent, goal, handle, gauge, lives) with a confidence and a one-action test. Every role was right on both frames.
- **Fused Briefing** (25/28) uses the Blueprint as its picture, states the Visual Routines and Look-alike Threads results as measured facts, and adds the Role Ledger guesses with their tests. Foresight Strip was dropped.

Exploded Inventory and Surprise Map did no better than the Legacy Encoder, at a tenth of its text, and were dropped as stand-alone presentations. What moved understanding most were the computed relations of Visual Routines and Look-alike Threads, which state facts a model cannot reliably read off a picture, and the Role Ledger, which gives the fastest gist. The Fused Briefing reached 25/28 at about 370 tokens per frame, 8.3× the Legacy Encoder's score with 89.4% fewer text tokens. Two caveats apply. The grader was the same agent that built the presentations and knew both games, and no blind reader was used. The Fused Briefing was designed after the other eight were graded and was graded on the same two frames, so its score is optimistic.

4.3 Held-Out Check: Facts Versus Guesses

We then checked the presentations on the 23 other official games, none of which had been used in the design. This check was graded by the engine rather than by hand (Table 2). Turning a screenshot into a grid generalised perfectly (23/23 games at 100% cell accuracy). The guesses did not: they had effectively been fitted to the two test frames and were right about half the time or less on unseen games.

Check on the 23 held-out games	Result
Screenshot to grid, exact cells	23/23 games
Agent identity, keyboard games	6/14 games
Moves-left gauge found	10/20 games
Click handles that respond	7/29 handles
Keyboard foresight fully right	1/16 games

Table 2: Held-out check of the single-frame guesses. For click handles, the guesses were good on `dc22` (2/2), `sb26` (4/4) and `ka59` (1/1) and bad on `lp85` (0/17). On two further keyboard games nothing moved on the first key press, so the agent-identity guess could not be checked.

This fixed the design rule of the engine: *computed relations (reachability, enclosure, tethers and midpoints, alignment, shape equivalence) are stated as facts, because they are true by construction, whereas roles and dynamics from a single frame are stated as guesses, each with a one-action test that settles it*. One real action answers what a single-frame prior cannot. We did not run the Fused Briefing as a whole on the held-out games, and the lab reports no RHAE. The end-to-end effect of the engine on the agent is not isolated by an ablation: the trajectory of Section 5.1 changes several components at once.

4.4 The Perception Engine

The final engine is the Fused Briefing, rewritten to work directly on the grid inside the agent. It is about 1,500 lines of host-side NumPy and Pillow with no learned parameters. Over 225 real frames (the first frame and eight random actions on each of 25 offline games) it raised no errors, with a median of 42 ms for the briefing and 40 ms for the picture (maximum 0.38 s for a briefing). For every environment transition it generates three synchronized representations sharing one persistent object identity space (Figure 5, Algorithm 1).

View 1: Morphological Segmentation, Tracking, and Change Ledgers. The background canvas is identified via the modal color index across the frame. All remaining 4-connected monochromatic components are segmented into discrete entities characterized by spatial bounding boxes, exact pixel masks, adjacency matrices, and topological containment hierarchies. On the initial frame of a level, entities are assigned persistent identifiers in standard raster reading order. On subsequent frames, an invariant bipartite matching algorithm tracks entity persistence across five sequential priority passes:

1. *Rigid Invariance*: Identical shape hash, color index, and spatial coordinates.
2. *Spatial Translation*: Identical shape hash and color index with translated coordinates.
3. *State Recoloring*: Identical shape hash and spatial coordinates with altered color index.

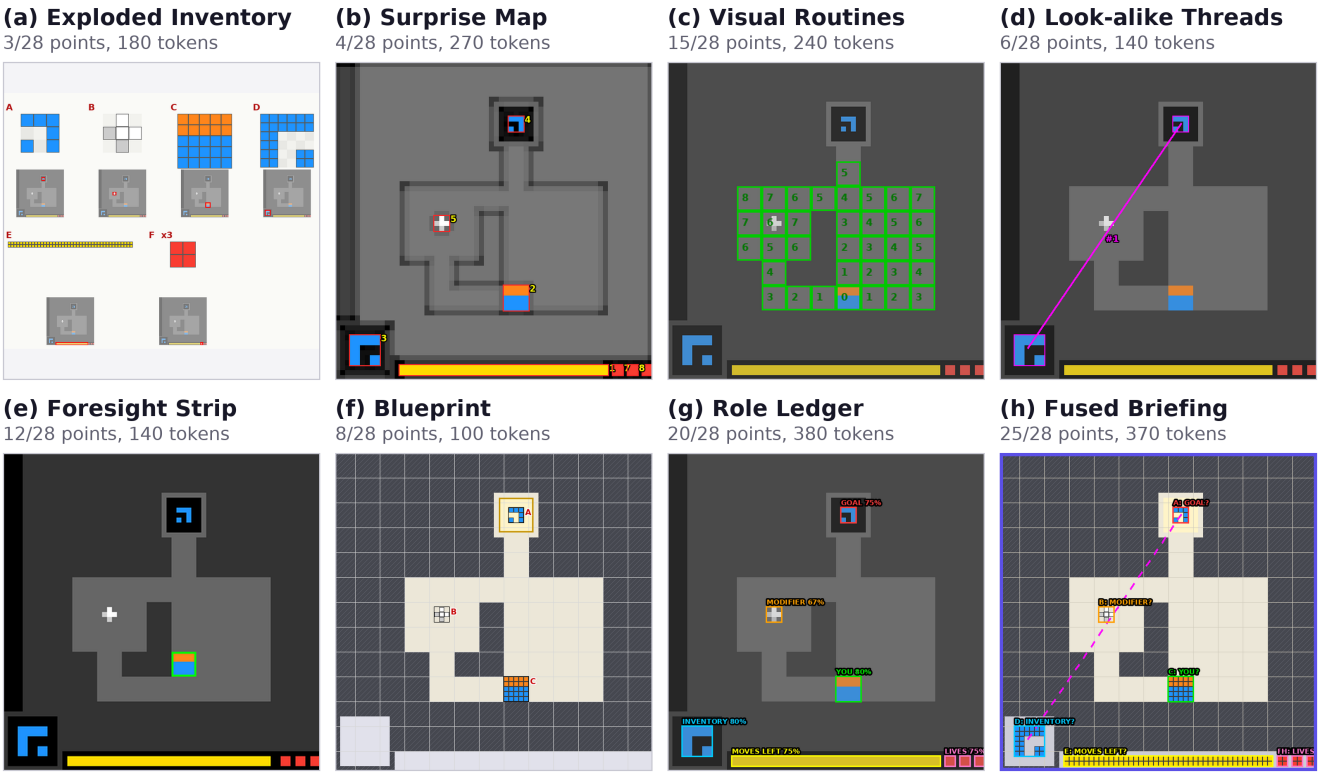


Figure 3: The image part of each candidate presentation for the opening frame of `ls20`, with its rubric score and its text cost per frame. (a) Exploded Inventory pulls out every object and stacks copies. (b) Surprise Map keeps colour only where the frame is unpredictable. (c) Visual Routines marks reachability, with the step count to each tile. (d) Look-alike Threads joins shapes that match under rotation, mirroring or scale. (e) Foresight Strip imagines the next frame (here, one step up). (f) Blueprint redraws floor, walls and lattice as a clean schematic. (g) Role Ledger tags each object with a guessed role and a confidence. (h) Fused Briefing, the presentation we kept, combines the blueprint with measured facts and tagged guesses.

4. *Morphological Deformation*: Identical color index with significant spatial overlap ($> 50\%$ IoU).
5. *Novel Component Instantiation*: Entities unmapped by preceding passes are instantiated with fresh unique IDs.

Consequently, symbolic references such as `objects[10]` within the agent's Python kernel refer to the identical physical entity across the entire level lifetime. The spatial grid is transcribed into a lettered alphanumeric matrix with explicit row and column headers. Sequential actions append a differential change ledger formatted as: "`#12 A4(right): 6 cells altered at r10-12 c20-25; HUD altered (1 cell)`". Environmental changes within the peripheral 3-cell screen boundary are partitioned into HUD telemetry, preventing minor status updates (e.g., action counter decrements) from obfuscating playfield invariants.

View 2: Layout Decomposition and Verified Geometric Invariants. The structural briefing decomposes the playfield into macroscopic functional layouts. Color partitions spanning $> 8\%$ of total frame area are classified as terrain. The remaining visual structure is decomposed into functional primitives: linear corridors, peripheral boundary gauges, composite multi-colored assemblies (clustered via union-find over 8-connected components), panel icons, and dotted figures.

Conditioned on this layout, the engine computes seven categories of mathematically verifiable MEASURED INVARIANTS, true by geometric construction:

- **Lattice Reachability**: Breadth-first search (BFS) over detected spatial lattices computes exact traversal distances from potential agent entities to all interactive targets (e.g., "`obj 6 reachable in 6 steps`"; "`enclosure entry accessible at bottom face in 5 steps`").
- **Topological Enclosure**: Detection of closed boundaries enclosing empty spatial cavities, mapped against entities matching those cavity dimensions.
- **Midpoint Tethers**: Identification of linear linkages spanning entity pairs, calculating mechanical centerpoints (e.g., "`obj 25 held at midpoint of obj 16 and 36; endpoint translation forces midpoint translation`").
- **Coordinate Collinearity**: Verification of shared row or column alignments between key landmarks.

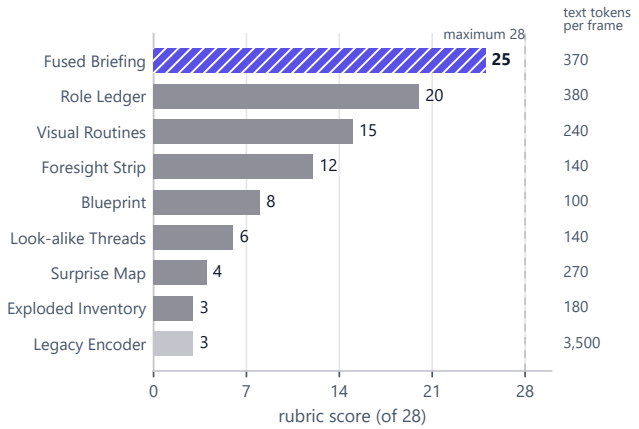


Figure 4: Rubric score (of 28) and text tokens per frame for the Legacy Encoder and the eight presentations, on the two test frames. The Fused Briefing scores 25/28 against 3/28 for the Legacy Encoder at about a tenth of the text.

- **Dihedral Affine Symmetries**: Identification of shape equivalence under the dihedral group $D_4 \times \mathbb{Z}^+$ (spanning reflection, 90° rotations, and integer scaling), identifying cross-scale functional analogues (e.g., "`obj 12 is a 2× scaled reflection of goal obj 4`").
- **Multiplicity Counters**: Clusters of identical morphological instances, parsed as discrete countable sets (e.g., life indicators, consumable resources).
- **Color Polarity Asymmetries**: Detection of morphologically symmetric entities exhibiting asymmetric chromatic distributions, marking directional orientation or active operational state.

Affordance Role Induction and Minimal Falsification Probes. Subsequent to invariant verification, non-terrain components are evaluated against eight canonical game-design affordance roles via weighted evidence scoring (Table 3). To eliminate the risk of hallucinated mental models, the engine couples each inferred role hypothesis with an explicit, single-action falsification probe:

The natural language briefing is bounded to 30 lines by default, allocating at most one-third of that budget to role hypotheses and sharing the rest among the invariant categories so that no single kind of fact can fill it; facts touching the guessed agent and goal come first, and the remainder are counted ("`+N more facts`") and stay available through `observe().briefing`. Complete structured briefings are provided in Appendix A.

View 3: Structured Affordance Map. The spatial map (Figure 5c,d) renders abstracted semantic structure rather than raw pixels: accessible terrain is rendered in pale tones, impassable walls are hatched, detected lattice grids are overlaid as fine lines, and all entities are bordered by role-colored bounding markers labeled "`id: ROLE?`". Look-alike symmetries under $D_4 \times \mathbb{Z}^+$ are joined via dashed chromatic threads, while mechanical tether linkages display midpoint concentric rings. Peripheral HUD panels are isolated and magnified on the right. Crucially, natural language annotations are excluded from image pixels and transmitted strictly as structured text, maximizing token efficiency and eliminating visual OCR errors.

Affordance Role	Evidentiary Priors (Scored)	Unit Falsification Probe
Player / Agent	Compact, multi-chromatic, unique, lattice-aligned	Issue 1 directional action: verify 1-tile translation
Target Goal	Hollow receptor, bounding enclosure, HUD shape matching	Transport matching entity into enclosure
Action Gauge	Elongated peripheral bar spanning screen margin	Issue arbitrary action: verify 1-unit reduction
Life Counter	Multiple identical glyphs in peripheral HUD	Observe count upon episode reset or hazard
Inventory	Isolated HUD glyph mirroring playfield entity	Observe glyph after interacting with modifier
Modifier	Compact symbol exhibiting chromatic asymmetry	Traverse onto entity: verify inventory mutation
Actuator Handle	Tether endpoint; chromatic toggle pair	Issue coordinate click: verify spatial jump
Coupled Cargo	Mechanically constrained between two tether handles	Translate handle: verify proportional shift

Table 3: Affordance role induction rules. Each hypothesis is presented with evidentiary confidence and an explicit single-action falsification test, ensuring erroneous hypotheses are refuted at minimal action cost.

Observation Scheduling. The full multimodal observation (briefing text, alphanumeric grid, and rendered map) is transmitted at level inception, following context window purges, and when the agent calls `observe()` after acting, at most once per state. Following intermediate actions, the agent receives an abbreviated observation: the differential change ledger, a cropped alphanumeric window centered on the modified coordinate region with a 3-cell margin, and the updated visual map, consuming fewer than 600 tokens. The complete object table is retained host-side and queried on-demand via Python kernel calls (`observe().objects`).

Hardening inside the agent. Two problems appeared only once the engine ran on live play. First, the test for a tile lattice flipped during play: `ls20` lost its 5×5 grid after one move, because the gauge in the status box added off-grid edges, and `ka59` lost its 3×3 grid whenever the agent touched another object. Status boxes are now left out of the test and the level's previous lattice is kept while it still fits, which removed all flips on the 225 frames. Second, "X is directly above Y" facts crowded out everything else: on `tr87` they filled 27 of the 30 briefing lines (68 facts in all, and 792 on `bp35`). A fixed cap also deleted facts from `observe().briefing`, so we kept every alignment fact, put those touching the guessed agent and goal first, and let each kind of fact take only its

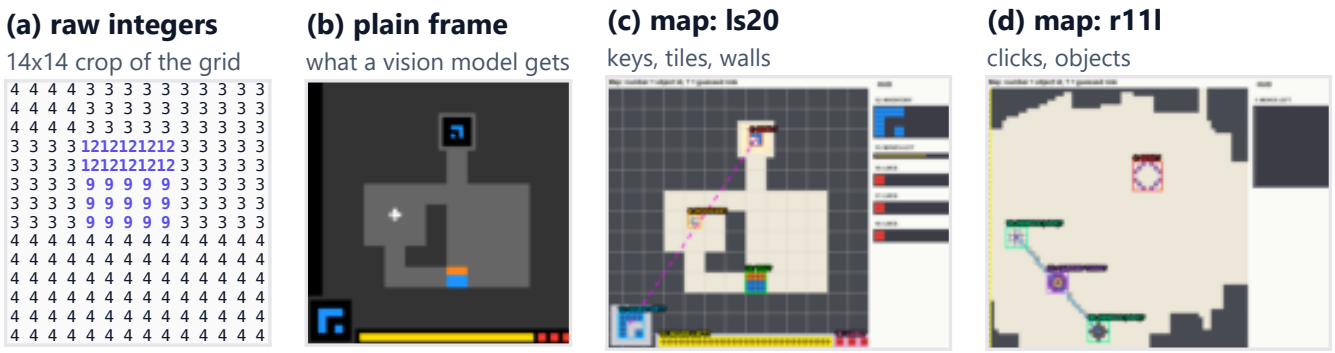


Figure 5: Structural multimodal representation of an ARC-AGI-3 frame produced by the perception engine. (a) Conventional serialization provided to language models: an unparsed matrix of discrete integer tokens (14×14 crop of a 64×64 frame). (b) Rendered 2D visual frame. (c, d) Affordance-annotated perception maps generated for the opening frames of `ls20` (discrete keyboard controls) and `r11l` (continuous spatial clicks). Traversal floor surfaces are rendered in light tones, impenetrable barriers are hatched, and peripheral HUD panels are isolated and scaled on the right. Tracked entities display persistent integer IDs alongside scored affordance roles. Magenta dashed threads indicate dihedral shape equivalence under rotation and scaling; yellow rings indicate mechanical midpoint tethers.

share of the lines shown.

4.5 The Overall Sarbloh Agent

Figure 6 puts the perception engine in the context of the whole agent. The runtime orchestrates 25 concurrent game sessions multiplexed across a single GPU. For each environment, the host sandbox intercepts the raw frame and the perception engine turns it into the aligned views described above. An append-only context manager streams these views into the model prompt while preserving prefix-cache residency. The reasoning backbone works through an embedded IPython kernel, modelling the environment in Python and executing multi-step paths by speculative execution. The remaining components are described below.

Speculative code-as-action. Sarbloh acts exclusively through a single persistent IPython execution kernel, extending the CodeAct paradigm (Wang et al. 2024). Within the execution REPL, `observe()` returns the current multimodal observation state as Python data structures. Action commitment is performed exclusively through a speculative execution primitive:

```
await play(path, predict=transition_model)
```

where `path` denotes a planned sequence of up to 100 candidate actions, and `predict` specifies an optional programmatic world model $\hat{f}(s,a) \to \hat{s}$. In preliminary iterations, providing the model with separate, distinct tools for code execution and action submission induced severe mode-switching loops, where the agent vacillated between code generation and single-step action queries. Unifying action commitment directly into the Python kernel enabled the model to synthesize algorithmic state machines, execute trajectory searches (BFS/DFS), and commit complete planned trajectories within a single execution turn.

Algorithm 2 defines the speculative execution contract. Prediction discrepancies between the agent's internal model $\hat{s}$ and the true environment state s' do not terminate execution prematurely; rather, discrepancies are registered with exact coordinate diffs, triggering a shortened thinking budget on the subsequent turn to facilitate rapid local hypothesis refinement.

Conversely, speculative execution is immediately preempted whenever runtime telemetry confirms that further action commitment is deleterious. The *state revisit leash* monitors the hash of playfield configurations (excluding the peripheral 3-cell HUD border): if the agent reproduces a state visited earlier within the current attempt, the execution loop is immediately aborted, arresting cyclical wandering. Furthermore, voluntary episode resets are bounded to three per level, eliminating pathological restart loops observed in unconstrained baselines. When evaluated post-hoc across offline trajectories, the revisit leash successfully suppressed 2,331 redundant exploratory actions across 9,913 total steps without truncating a single successful solution path.

Monotonic append-only context. Our deployed backbone architecture integrates Gated DeltaNet linear recurrent layers with sparse multi-head attention (Yang, Kautz, and Hatamizadeh 2024). While linear attention provides exceptional generation speed, its recurrent state representation imposes a rigid operational invariant: the model serving engine can only resume generation from state checkpoints recorded at the terminal boundary of prior requests. Any retrospective modification to historical tokens—such as replacing stale visual images or updating historical messages—invalidates the recurrent state, triggering expensive full prefill recomputation.

In early deployments, unconstrained prompt rewriting resulted in catastrophic cache degradation: 3,283 of 5,599 sequential requests experienced 0% prefix cache reuse, requiring continuous prefill that saturated GPU bandwidth. We resolved this systems bottleneck by enforcing a strictly *monotonic, append-only context protocol*. Generation histories re-

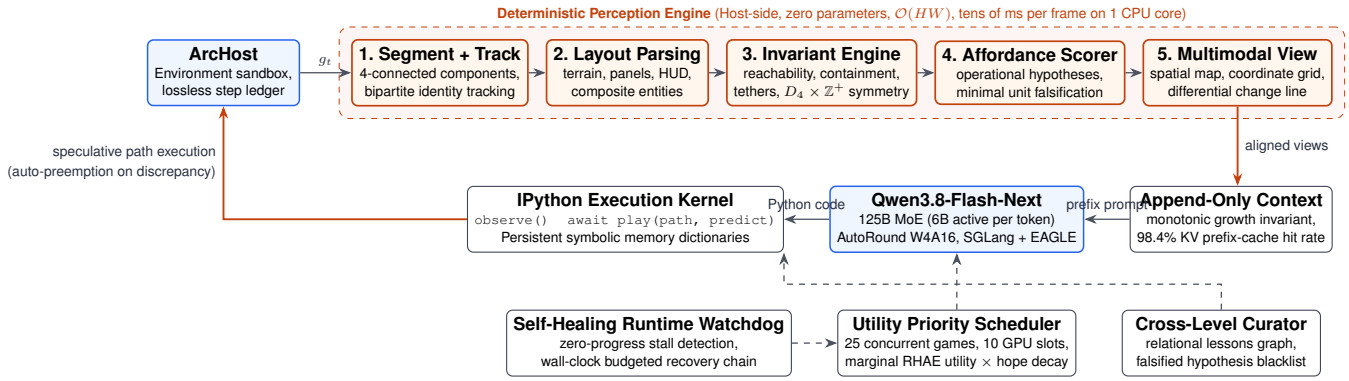


Figure 6: System architecture of SARBLOH. The host runtime encapsulates the environment sandbox. Each observation frame is processed by the deterministic perception engine (orange), generating three aligned modalities ingested into a strictly monotonic, append-only context window. The agent reasons exclusively by executing code within an embedded IPython kernel, inspecting state via `observe()` and committing speculative trajectories via `await play()`. Across 25 concurrent game sessions, a single enterprise GPU is dynamically multiplexed via an optimal utility-based priority scheduler.

tain all preceding reasoning traces, newly rendered images are appended as discrete immutable messages following the action output that produced them, and state modifications are appended strictly as delta logs. As demonstrated in Table 4, this append-only invariant raised prompt prefix-cache utilization to **98.4%** across full nine-hour runs, effectively rendering the serving system decode-bound.

Deterministic context drainage. When cumulative context length surpasses 118k tokens, the runtime executes a deterministic context drainage routine, discarding the oldest conversational turns in code until total length falls below 55k tokens. Unlike prior agent frameworks that rely on recursive model-generated summaries (Knutsen and Klein 2026), our purge routine uses zero LLM calls, completely avoiding the accumulation of hallucinated summaries. The initial system prompt is permanently pinned, and the active symbolic memory state and Python code graph are re-injected as a clean prefix checkpoint (Algorithm 3, Line 3).

Working memory and relational graph. The agent maintains working memory within four structured dictionaries persisted inside the IPython kernel: `GOAL`, `PLAN`, `HYPOTHESES`, and `FINDINGS`. These structures are retained across level transitions, ensuring that inductive knowledge acquired in early levels remains accessible when solving complex compositional stages. A fifth structure, `SKILLS`, is populated via a post-level *curator*. Following each completed level, a fast structured JSON call evaluates verified versus refuted hypotheses, updating a cross-level relational lessons graph and maintaining an explicit blacklist of falsified rules to prevent the re-generation of refuted behavioral hypotheses. Live abstract syntax tree (AST) analysis inspects kernel variables following each cell execution, appending a single telemetry line summarizing defined or broken functional dependencies.

Model quantization and serving runtime. SARBLOH deploys Qwen3.8-Flash-Next, an advanced 125B parameter Mixture-of-Experts (MoE) architecture activating 6B parameters per token, featuring hybrid Gated DeltaNet and sparse attention layers. The model is quantized to W4A16 precision using AutoRound (Cheng et al. 2023) and served via SGLang (Zheng et al. 2023) with EAGLE speculative decoding (Li et al. 2024b) (configured with 3 draft speculative tokens), FP8 KV caching, and a 136k context window. Each game turn is allocated a 12k token generation ceiling. A self-healing watchdog monitors serving process health: servers that experience thread hangs (detected when token generation counters cease incrementing despite active client requests) are immediately terminated via stack dumps and restarted along a staged profile fallback chain, with restart budgets metered strictly by remaining wall-clock execution time.

Utility-driven priority scheduler. All 25 games are instantiated concurrently at runtime start, but active GPU access is restricted to 10 concurrent serving slots to prevent KV-cache thrashing. Slot allocation is governed by a dynamic utility-driven priority function. For a game session currently evaluating level $L+1$ of n, having expended t generation tokens and a actions on the active level, its priority index π is computed as:

$$\pi = \left(\frac{L+1}{\frac{n(n+1)}{2}} \cdot \left(\frac{h}{h+a} \right)^2 + \gamma_{\text{tail}} \right) \cdot 2^{-\left(\left(\frac{t}{\tau}\right)^2 + \left(\frac{a}{\alpha}\right)^2 \right)}, \quad (3)$$

where $h = 25$ represents the median human baseline constant, $\tau = 40$k tokens denotes the token exhaustion threshold, $\alpha = 115$ actions denotes the exploration action limit, and $\gamma_{\text{tail}} = 0.05$ ensures non-zero starvation prevention. The first factor formalizes the expected marginal RHAE reward attainable upon clearing the active level, discounted quadratically by accumulated action expenditure. The second factor models an exponential *progress hope function*: games that consume substantial compute or action budgets without achieving state transitions experience rapid priority decay, gracefully yielding GPU resources to productive game sessions. Upon clearing a level, a game's counters are reset and its priority is immediately re-evaluated.

Algorithm 1: PERCEIVE: Deterministic Frame-to-Modality Mapping

Input: Raw grid $g_t \in \{0..15\}^{H \times W}$, persistent tracker T, lattice prior Λ', legal actions $\mathcal{A}$
Output: Structured briefing B, tracked entities O, visual map M, differential change δ

$O \leftarrow \textsc{MorphologicalSegmentation}(g_t)$;
$(O, \delta) \leftarrow T.\textsc{BipartiteMatch}(O)$; ▷ *temporal identity persistence*
$\mathcal{T} \leftarrow \{c \in \{0..15\} : |g_t{=}c| > 0.08\,HW\}$; ▷ *terrain segmentation*
$L \leftarrow \textsc{ExtractLayout}(g_t, \mathcal{T})$;
foreach *entity* $e \in L$ **do**
  $e.is_hud \leftarrow \textsc{IsPeripheralBoundary}(e)$;
  $e.id \leftarrow \textsc{MajorityOverlapId}(e, O)$;
$e_{\text{agent}} \leftarrow \textsc{IdentifyCandidateAgent}(L, \mathcal{A})$;
$\Lambda \leftarrow \textsc{DetectLattice}(L)$ **or** Λ';
$F \leftarrow \textsc{BFSReach}(e_{\text{agent}}, \Lambda) \cup \textsc{Enclosures}(L)$;
$F \leftarrow F \cup \textsc{Tethers}(L) \cup \textsc{Symmetries}_{D_4 \times \mathbb{Z}^+}(L)$;
foreach *entity* $e \in L$ **do**
  $\boldsymbol{\sigma}_e \leftarrow \textsc{ScoreAffordances}(e, F, \mathcal{A})$;
  $\hat{\rho}_e \leftarrow \arg\max_r \boldsymbol{\sigma}_e[r], \quad p_e \leftarrow \boldsymbol{\sigma}_e[\hat{\rho}_e]/(1 + \sum_r \boldsymbol{\sigma}_e[r])$;
$B \leftarrow \textsc{FormatInvariants}(F)$;
$B \leftarrow B \,\|\, \textsc{FormatHypotheses}(\hat{\boldsymbol{\rho}}, \boldsymbol{p}, \textsc{Tests}(\hat{\boldsymbol{\rho}}))$;
$M \leftarrow \textsc{RenderAffordanceMap}(L, \hat{\boldsymbol{\rho}}, F)$;
return $(B, O, \textsc{ToAlphanumeric}(g_t)), M, \delta$

Algorithm 2: PLAY: Speculative Path Execution with Runtime Preemption

Input: Action trajectory $m_1, \ldots, m_K$, optional transition model $\hat{f}$

$s \leftarrow \textsc{Observe}()$; *discrepancy* $\leftarrow \varnothing$;
for $i \leftarrow 1$ **to** K **do**
  if $\hat{f} \neq \varnothing$ **then** $\hat{s} \leftarrow \hat{f}(s.\text{board}, m_i)$;
  $r \leftarrow \textsc{ArcHost.CommitAction}(m_i)$; ▷ *verified execution*
  $s' \leftarrow \textsc{Observe}()$;
  if $\hat{f} \neq \varnothing$ ***and*** *discrepancy* $= \varnothing$ ***and*** $\textsc{Diff}_{\neg HUD}(\hat{s}, s'.board) \neq \varnothing$ **then**
    discrepancy $\leftarrow (i, m_i, \text{mismatched_cells})$;
    $\textsc{ArcHost.SetShortThinkingBudget}()$;
  if *level_completed* ***or*** *game_over* ***or*** s' *visited in current attempt* ***or*** *barrier hit twice* **then**
    break;
  $s \leftarrow s'$;
return *change_ledgers, discrepancy,* s'

Algorithm 3: SESSION: Turn Execution Loop for an Individual Game

while *execution budget active* ***and not*** *terminal_state* **do**
  for $j \leftarrow 1$ **to** *12* **do**
    if $|context| > 118k$ **then** drain oldest turns to 55k; re-insert pinned memory;
    $y \leftarrow \text{LLM}(\text{context})$; ▷ *98.4% prefix-cached*
    execute IPython code cells in y; append execution output, new visual map, state diffs;
    if *action committed to host* ***or*** *thinking tokens* $> 16k$ **then break**;
  $\textsc{Scheduler.YieldSlot}()$; ▷ *yield GPU slot after quantum*
  if *level_completed* **then** $\textsc{CurateKnowledge}()$; update skill graph;

5 Results and Empirical Evaluation

We evaluate SARBLOH across three rigorous empirical dimensions: (i) official evaluation trajectory on the undisclosed Kaggle ARC-AGI-3 benchmark, (ii) per-game comparison with five frontier models and a hardware-matched control run of the Milestone #1 Duck harness on identical enterprise hardware, and (iii) systems execution telemetry quantifying computational and token efficiency under our single-GPU budget.

5.1 Evaluation Trajectory on Hidden Benchmark

The official Kaggle ARC Prize track evaluates submitted notebooks on held-out, undisclosed game environments within an offline execution sandbox, providing an uncontaminated measure of out-of-distribution reasoning and generalization.

Our initial architectural baseline—a direct port of the Prime Agent framework (Karten et al. 2026) operating with open-weight models (Gemma-4-31B and Qwen3.8-27B)—achieved scores between 0.00 and 0.27. Detailed trace diagnostics revealed that while the model successfully generated Python syntax, it engaged in unguided brute-force search: across a 10-game evaluation, the agent initiated over 350 voluntary resets without successfully identifying a single goal state.

Systematic architectural interventions drove continuous, measurable performance gains across successive builds:

1. **Serving and Backbone Upgrade**: Migrating the inference engine to Qwen3.8-Flash-Next on SGLang, optimizing prompt prefix-caching, and enabling concurrent multi-game execution elevated the official score to **5.14**.
2. **Tool Unification and Context Invariants**: Merging action dispatch directly into the Python kernel (`await play()`) and replacing lossy summarization with deterministic context draining increased performance to **5.65** and **6.06**.
3. **Symbolic Memory and Priority Scheduling**: Introducing cross-level memory dictionaries, real-time AST code graph tracking, and utility-driven priority scheduling across concurrent sessions yielded a substantial leap to **10.98**.
4. **Integrated Multimodal Perception Core**: Integrating the complete multimodal perception engine (combining bipartite tracking, geometric invariants, and affordance hypothesis testing) alongside refined context guardrails established our peak verified score of **16.53**.

This trajectory represents a 13.7× efficiency improvement over the 1.21 score that achieved first place in Milestone #1

(June 2026), achieved over a 24-day engineering cycle. While top-tier submissions on the live public leaderboard have achieved higher aggregate scores through proprietary ensemble scaling and extensive pre-training corpora, SARBLOH demonstrates that disciplined perceptual grounding, monotonic KV-cache management, and speculative execution deliver unprecedented reasoning density within the strict resource envelope of a single enterprise GPU.

5.2 Hardware-Matched Comparative Benchmark against Milestone #1 Baseline

To rigorously isolate the algorithmic and systems contributions of our agent from underlying hardware capabilities, we conducted an extensive controlled benchmark against a recent release of the Tufa Labs Duck harness (Tufa Labs 2026)—the architecture that won Milestone #1. Both agents were deployed on an identical server platform featuring a single NVIDIA RTX PRO 6000 (96 GB VRAM) and identical SGLang runtime configurations serving the Qwen3.8-Flash-Next model. The Duck harness executed four concurrent passes per game (totaling 100 concurrent execution streams), whereas SARBLOH executed a single pass across the 25 public games governed by our dynamic priority scheduler.

Figure 7 plots the cumulative count of cleared levels as a function of wall-clock execution time. SARBLOH demonstrates a pronounced advantage in discovery speed throughout the entire nine-hour window:

- At $t = 1.0$ hour, SARBLOH clears 73 levels compared to 54 for the baseline (35.2% advantage).
- At $t = 2.0$ hours, SARBLOH clears 97 levels compared to 63 for the baseline (54.0% advantage).
- At $t = 4.0$ hours, SARBLOH clears 136 levels compared to 103 for the baseline (32.0% advantage).
- At the conclusion of the execution envelope ($t = 8.2$ hours), SARBLOH resolves **143** of the 183 total levels (78.1% completion), whereas the Duck harness completes 129 distinct levels across its four concurrent passes.

Normalized per GPU-hour, a single pass of the Duck harness (allocating approximately one-quarter of total GPU compute) reached approximately 104 levels. The primary performance advantage of SARBLOH stems from its utility-driven priority scheduler (Equation 3): rather than dividing GPU capacity uniformly across redundant copies of every game, our runtime dynamically concentrates compute on the specific game states exhibiting the highest expected marginal RHAE gains. In time-bounded evaluations, prioritizing productive exploratory paths accelerates the discovery of distinct level solutions.

Figure 1 (shown after the abstract) gives the per-game score on all 25 public environments, set against five frontier models (data in `data/Score.csv`). SARBLOH has the highest score of the six systems on 21 games and reaches 100% on 13 of them. It trails the best frontier model on four: `dc22` (39.6 versus 47.6), `sp80` (22.4 versus 60.1), `tn36` (7.7 versus 75.0) and `wa30` (26.3 versus 76.6), all of them games where GPT-6.1 Sol is the leader. On `g50t`, which no frontier model solves (best 0.28), SARBLOH scores 54.6.

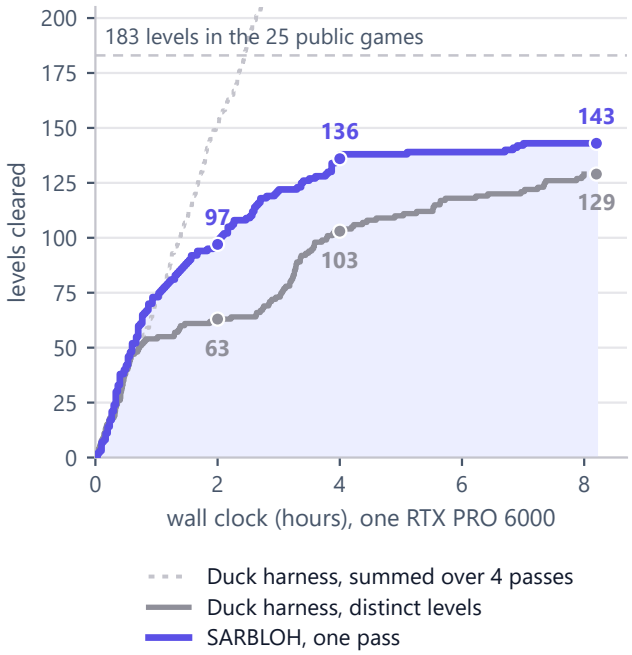


Figure 7: Cumulative level completion over wall-clock execution time across the 25 public ARC-AGI-3 environments on identical hardware (one NVIDIA RTX PRO 6000) and model weights. SARBLOH (single pass, utility scheduler) consistently outpaces the Duck harness baseline (best of 4 concurrent passes), resolving 143 of 183 levels in 8.2 hours compared to 129 distinct levels for the baseline.

The mean over the 25 games is 72.5 for SARBLOH against 54.8 for GPT-6.1 Sol, the strongest frontier model. These are public-game scores and are not comparable to the hidden-set score of Section 5.1.

5.3 Action Efficiency on Cleared Levels

Because RHAE penalizes superfluous exploration quadratically, raw level completion counts do not fully convey operational efficiency. We therefore quantify the action ratio a_l/h_l across all successfully cleared levels.

Across our primary offline evaluation run, the *median* action ratio achieved by SARBLOH on cleared levels was **0.77** relative to the human baseline. In accordance with Equation (2), this level of efficiency earns the maximum allowable reward cap ($S_l = 1.15$). This metric demonstrates that once SARBLOH correctly induces the underlying mechanics of a level, its execution-grounded planning engine operates with super-human efficiency: the agent synthesizes optimal trajectory plans in code and commits them directly via speculative path execution.

Decomposing the error budget relative to a theoretical maximum score across the 25 public environments reveals that of the 40.6 points lost from a perfect score:

- 12.6 points (31.0%) result from minor action overhead on completed levels where $a_l > 0.93\,h_l$.

Metric (9 h, 25 games, 1 GPU)	Value
Prompt tokens served from warm prefix cache	**98.4%**
Total prompt tokens / generated reply tokens	1.78 B / 14.4 M
Total model invocations / runtime failures	21,224 / 0
Concurrent active games per execution turn (mean)	11.8
Cumulative turn execution rate across all games	42.7 turns/min
Sustained decode throughput per request stream	76 tok/s
Turn wall-clock latency, median / 90th percentile	2.3 s / 30.2 s
GPU temporal occupancy: decode / prefill	50.1% / 1.3%
Proportion of generation tokens allocated to thinking	78.0%
Wall-clock time per cleared level (median)	9.2 min
Action ratio (a_l/h_l) on cleared levels (median)	**0.77**

Table 4: Comprehensive systems and hardware telemetry for SARBLOH during a continuous nine-hour evaluation across 25 games on a single NVIDIA RTX PRO 6000 GPU.

- 8.0 points (19.7%) result from 13 levels that were actively attempted but failed to terminate within the action budget.
- 19.5 points (48.0%) result from subsequent compositional levels that were never reached because the agent stalled on an antecedent stage.

This error decomposition confirms that the overwhelming majority of score loss is attributable to perceptual incomprehension on antecedent levels rather than planning inefficiency once a level is understood.

5.4 Systems Telemetry and Compute Utilization

Table 4 summarizes the comprehensive hardware and execution telemetry captured across a continuous nine-hour evaluation run on a single NVIDIA RTX PRO 6000 GPU.

The telemetry confirms that our systems architecture successfully achieves a *decode-bound execution profile*. Prefill operations consume merely 1.3% of total GPU temporal occupancy, directly resulting from our 98.4% prompt prefix-cache hit rate. In contrast, prior implementations lacking append-only invariants experienced a 10% cache hit rate, causing prefill queue starvation that stalled concurrent execution streams. Over the complete nine-hour evaluation window, the SGLang runtime processed 21,224 sequential model invocations without experiencing a single server fault, crash, or memory overflow.

5.5 Failure Mode Analysis and Ablations

Detailed trace audits across unresolved levels revealed two primary operational failure modes:

1. **Cyclic Action Stalls**: In early builds lacking the state revisit leash, four games approached within 2–15 actions of their 2,000-action ceiling and entered degenerate continuation loops, emitting repetitive "`Holding...`" tokens across approximately 14,000 idle turns and consuming 1.24 B cached prompt tokens without altering the environment state. Enforcing a hard ceiling of 400 turns per game alongside the state revisit leash effectively eliminated these degenerate loops.
2. **History Code Stubbing Negative Transfer**: In an exploratory ablation aimed at token conservation, historical code cells in the prompt context were replaced with single-line stub markers (e.g., "`# [code executed]`"). Rather than writing novel solution code, the small 6B-active model mirrored the stub in 97.7% of subsequent turns, completely collapsing planning capacity (0 levels cleared). Small open-weight models exhibit intense in-context imitation bias, necessitating the retention of full syntactic code traces.

6 Limitations and Future Work

Limitations. Several architectural limitations remain. First, while affordance hypotheses are coupled with unit falsification tests, the current perception engine evaluates hypotheses independently at each frame without maintaining a fully unified Bayesian belief distribution across multi-step episodes. Second, although the agent plans with super-human efficiency once a level is understood, antecedent failure on complex perceptual transitions remains the primary source of score degradation. Finally, our current results rely on general-purpose open-weight models without task-specific policy fine-tuning.

Future work. Promising research avenues include:

1. **Closed-Loop Belief Updating**: Formulating affordance induction as a dynamic partially observable belief state, where the empirical outcomes of unit falsification probes explicitly update prior belief weights across compositional levels.
2. **Offline Trajectory Fine-Tuning**: Applying parameter-efficient fine-tuning (LoRA (Hu et al. 2021)) directly on our database of verified, high-efficiency execution trajectories, optimizing syntactic code generation and world model synthesis.
3. **Sample-Efficient Reinforcement Learning**: Investigating offline reinforcement learning algorithms, such as Group Relative Policy Optimization (GRPO (Shao et al. 2024)) and balanced policy clipping, to optimize planning depth and action parsimony within our fixed computational envelope.

7 Conclusion

On an interactive benchmark with quadratic action penalties, an agent cannot pay for poor perception with trial and error. Our ablations show that language and vision-language models fail on ARC-AGI-3 mainly because unparsed token grids hide spatial and topological structure, not because they cannot reason about it.

extscSarbloh addresses this with a zero-parameter, host-side perception engine that states computed geometric relations as facts and role guesses as hypotheses with one-action tests. The agent plans over these views in a persistent Python kernel and commits multi-step paths that stop when the game disagrees with a prediction. An append-only context keeps the prompt prefix-cache hit rate at 98.4% across 21,224 model calls on a single RTX PRO 6000. The agent resolves 143 of 183 public levels in 8.2 hours and scores

16.53 on the official hidden Kaggle evaluation, 13.7$imes$ the Milestone #1 benchmark.

A Perception Engine Structural Briefings

Below we provide the verbatim structured briefing text emitted by the deterministic perception engine for the opening frames of `ls20` and `r11l` illustrated in Figure 5. The object identifiers correspond exactly with the annotated spatial map and the kernel's symbolic state dictionary (`observe().objects`).

```
BRIEFING (arrow keys)
MEASURED (certain):
- Floor is a 5x5 tile grid.
- obj 4 (blue 6-cell shape): sits inside a black box
    that is NOT plain floor; the floor reaches the box's
     bottom side in 5 steps (the only way in)
- obj 6 (white/light grey plus): reachable in 6 steps
- obj 4 (blue 6-cell shape) is directly above obj 10 (
    blue/orange square) (shared columns 35-37)
- obj 12 (blue irregular shape (24 cells)) (drawn 2x
    bigger), mirrored left-right OR turned 90 deg
    clockwise, becomes exactly obj 4 (blue 6-cell shape)
- 3 identical red square (2x2): obj 16, 17, 18 -> a
    count of something (lives, items, tiles)
- obj 6 (white/light grey plus) has a symmetric shape
    but lopsided colours (marks a direction/state)
- obj 10 (blue/orange square) has a symmetric shape but
    lopsided colours (marks a direction/state)
GUESSES (check with one action):
- obj 10 (blue/orange square) = YOU (80%) - test: press
    one arrow key: it should move one tile and the gauge
     should drop
- obj 12 (blue irregular shape (24 cells)) = INVENTORY
    (80%) - test: watch it after touching the modifier
- obj 4 (blue 6-cell shape) = GOAL (75%) - test: get the
     right thing into it and see if the level ends
- obj 15 (yellow bar) = MOVES LEFT (75%) - test: take
    any action and check it shrinks by one unit
- obj 16, 17, 18 (red square) = LIVES (75%) - test:
    watch it when the gauge runs out or you fail
- obj 6 (white/light grey plus) = MODIFIER (67%) - test:
     step onto it, then check whether the inventory icon
     changed
```

```
BRIEFING (clicks)
MEASURED (certain):
- no tile grid -> free movement. Open space = black (81%
     of screen).
- all 4 main objects share one open space; straight
    paths between them are ALL CLEAR of rock
- the grey areas are not open floor (likely walls)
- obj 4 (purple dotted ring) encloses an EMPTY 5x5 space
     -- exactly the size of: obj 16 (white/purple
    diamond), obj 25 (purple/magenta disc), obj 36 (dark
     grey/purple diamond)
- a light grey line connects obj 16 (white/purple
    diamond) to obj 25 (purple/magenta disc)
- a light grey line connects obj 25 (purple/magenta disc
    ) to obj 36 (dark grey/purple diamond)
- obj 25 (purple/magenta disc) sits at the MIDPOINT of
    obj 16 (white/purple diamond) and obj 36 (dark grey/
    purple diamond) (error 0.5 cells) -> it is held
    between them; moving either end should move it to
    the new midpoint
- obj 16, 36 (white/purple diamond; dark grey/purple
    diamond) have the same shape in different colours (
    white/purple vs dark grey/purple)
GUESSES (check with one action):
- obj 4 (purple dotted ring) = GOAL (80%) - test: get
    the right thing into it and see if the level ends
- obj 16, 36 (white/purple diamond; dark grey/purple
    diamond) = HANDLE (click) (80%) - test: click an
    empty spot once: the selected handle should jump
    there
- obj 1 (white bar) = MOVES LEFT (75%) - test: take any
    action and check it shrinks by one unit
- obj 25 (purple/magenta disc) = CARRIED THING (75%) -
    test: move one handle: it should slide to the new
    midpoint
```

B System Configuration and Hyperparameters

Table 5 documents the primary execution parameters governing the scored benchmark evaluation.

System Configuration Parameter	Operational Value
Concurrent games / Active GPU slots	25 games / 10 slots
Scheduler quantum allocation	20 model calls per round
Action allocation ceiling per game	2,000 actions
Maximal steps per `play()` path	$\leq$100 actions
Turn reply budget / Inner requests	12k tokens / 12 calls
Turn preemption thinking threshold	16k tokens
Context drain threshold / target	118k tokens / 55k tokens
Context maximum window size	131,072 tokens
Visual map rendering scale	10 pixels per grid cell
Briefing line limit / Short-view ceiling	20 lines / 600 tokens
Symbolic memory caps (G/P/H/F)	150 / 200 / 400 / 300 tokens
Sampling temperature / top-p / top-k	$T = 0.6$ / $p = 0.95$ / $k = 20$
Serving engine and precision	SGLang, W4A16, FP8 KV cache
Speculative decoding configuration	EAGLE (3 speculative draft steps)

Table 5: System hyperparameter settings deployed in the official evaluation runtime.

C Artifacts and Reproducibility

All empirical findings, telemetry traces, and figures in this manuscript are fully reproducible using the scripts in `Writing/scripts/`:

- `collect_data.py`: Ingests the Kaggle submission log, per-game benchmark tables, and perception lab evaluation rubrics.
- `extract_curves.py`: Extracts wall-clock level completion trajectories from raw execution logs.
- `make_perception_figure.py`: Executes the host-side perception engine on raw game environments to

generate segmented objects, spatial maps, and structured briefings.

- `plot_results.py`: Generates all publication-grade figures in vector PDF and high-resolution raster formats.

All code, model serving scripts, evaluation environments, and experimental logs are publicly accessible under the Apache-2.0 license at https://github.com/Harsidak/SARBLOH.